Enterprise App & Service Principal Governance (App Registrations, API Permissions) — Government (GCC High)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Enterprise App & Service Principal Governance (Government / GCC High) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 5

Outcome . . . 5

1. Solution Overview — Enterprise App & Service Principal Governance (Government / GCC High) 5

The Entra application model . . . 5

Delegated vs. application permissions . . . 5

Consent — who approves the permissions . . . 6

The over-permission problem . . . 6

Secret and certificate hygiene . . . 6

What you can control . . . 6

Commercial vs GCC High — what's different . . . 7

Benefits . . . 7

Limitations (be honest with buyers) . . . 8

The standard deployment model . . . 8

2. Requirements Guide — Enterprise App & Service Principal Governance (Government / GCC High) 8

2.1 GCC High tenant . . . 8

2.2 Entra ID licensing (Government SKUs) . . . 8

2.2a Confirm US Gov feature parity for anything you layer on . . . 9

2.3 Administrative roles . . . 9

2.4 Consent policy decision . . . 9

2.5 App inventory approach . . . 9

2.6 Governance scope group (pilot) . . . 10

2.7 Microsoft Graph PowerShell SDK (US Gov) . . . 10

2.8 Break-glass accounts . . . 10

2.9 Network egress to US Gov endpoints . . . 10

Checklist . . . 11

3. Implementation Guide — Enterprise App & Service Principal Governance (Government / GCC High) 11

Step 0 — Confirm you are in the sovereign cloud . . . 11

Step 1 — Create the governance scope and reviewer groups . . . 12

Step 2 — Inventory every enterprise app and service principal . . . 12

Step 3 — Review API permissions and consents for over-privilege . . . 12
Step 4 — Configure the user-consent policy . . . 12
Step 5 — Enable the admin-consent workflow . . . 13
Step 6 — Remediate risky grants (staged) . . . 13
Step 7 — Manage secret and certificate expiry . . . 13
Step 8 — Validate on the pilot . . . 14
Step 9 — Expand tenant-wide . . . 14
5. Testing Guide — Enterprise App & Service Principal Governance (Government / GCC High) . . . 14
Test 0 — You are in the sovereign cloud . . . 15
Test 1 — Inventory is complete . . . 15
Test 2 — Over-permission report is accurate . . . 15
Test 3 — User-consent policy is set . . . 15
Test 4 — Consent-phishing is blocked . . . 15
Test 5 — Admin-consent workflow works . . . 15
Test 6 — Risky grant remediation (preview then apply) . . . 15
Test 7 — Credential expiry surfaced . . . 15
Pass/fail gate . . . 15
6. Troubleshooting Guide — Enterprise App & Service Principal Governance (Government / GCC High) 16
A. Wrong cloud / GCC High connection problems (check this first) . . . 16
B. Discovery or report returns nothing / errors . . . 16
C. Unowned or unknown apps . . . 17
D. A legitimate app broke after remediation . . . 17
E. User-consent policy not taking effect . . . 17
F. Admin-consent requests not reaching reviewers . . . 18
G. Consent-phishing still succeeding . . . 18
H. A feature you saw in Commercial is missing in GCC High . . . 18
I. Diagnostics quick reference (US Gov) . . . 18
J. When to escalate . . . 18
7. Best Practices — Enterprise App & Service Principal Governance (Government / GCC High) . . . 19
8. Deliverables Summary — Enterprise App & Service Principal Governance (Government / GCC High) 20
Documentation . . . 20
PowerShell scripts (all connect `-Environment USGov` → `graph.microsoft.us`) . . . 20
What the buyer receives . . . 21
Outcome . . . 21
Appendix A — PowerShell Scripts 22

Executive Summary — Enterprise App & Service Principal Governance (Government / GCC High)

White-label · Entra ID Solutions · US Sovereign Cloud (GCC High)

Every SaaS integration and internal app you connect leaves behind three things in your tenant: an enterprise application, a service principal, and a set of API permissions. Over time they pile up — apps nobody owns, dangerous consented permissions, and app secrets or certificates quietly expiring or lingering forever. Attackers love them: an over-permissioned or consent-phished app is a quiet, persistent foothold that survives password resets and MFA. On a GCC High tenant handling CUI or ITAR data, that foothold is a controlled-data exposure.

This is the **Government (GCC High)** edition of a complete, deploy-it-today package for governing enterprise apps and service principals in Microsoft Entra, written for an administrator with no prior app-governance experience. It is authored for the **US sovereign cloud** — every portal path is the `*.us` variant (`entra.microsoft.us`, `portal.azure.us`, `portal.office365.us`), sign-in is `login.microsoftonline.us`, and every script connects to US Gov Microsoft Graph (`graph.microsoft.us`, `Connect-MgGraph -Environment USGov`). The Commercial-vs-GCC-High differences are called out where they matter, so you configure a government tenant correctly instead of running Commercial steps that silently fail.

What you get

• A plain-language Solution Overview of the Entra app model — app registrations vs. enterprise apps and service principals, delegated vs. application permissions, admin and user consent, the over-permission and consent-phishing risk, and the secret/certificate lifecycle — with an explicit **Commercial vs GCC High — what's different** section.

• A cited **GCC High parity reference** so you never build a step on a feature that is not in the sovereign cloud.

• A Requirements Guide with verify / implement-if-missing for every prerequisite, in US Gov terms.

• An Implementation Guide of precise, click-by-click steps using the GCC High portals, with the sovereign-cloud caveats inline at the step where they matter.

• Five PowerShell scripts for app and service-principal discovery, risky-consent and expiring-secret validation, an over-permission and expiring-secret report, safe remediation of risky grants, and a governance-scope group builder — all connecting to `graph.microsoft.us` via `-Environment USGov`.

• Testing and Troubleshooting guides for a confident, staged rollout, with GCC-High-specific gotchas.

What this delivers

A governed application estate on a sovereign tenant — every enterprise app and service principal inventoried, API permissions and consents reviewed for over-privilege, a user-consent policy that stops consent-phishing, an admin-consent workflow so users can request access safely, risky grants remediated, and app secrets and certificates on a managed lifecycle — proven on a pilot scope before you

touch the whole tenant.

The standard deployment model

Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

Every enterprise app and service principal inventoried and governed on GCC High — right portals, right endpoints, parity gaps handled — over-permissioned grants and risky consents remediated, user consent controlled, and app secrets and certs on a managed lifecycle — closing a favorite attacker foothold, validated on a pilot before you scale.

1. Solution Overview — Enterprise App & Service Principal Governance (Government / GCC High)

This is the **Government (GCC High)** edition of the Enterprise App & Service Principal Governance solution. The identity model, the risks, and the deployment method are the same as Commercial — but the portals, sign-in, and Microsoft Graph endpoints are the **US sovereign** `*.us` variants, and a few capabilities lag or differ. Where a GCC High specific is not confirmed in Microsoft's US Gov documentation, it is flagged **"verify for GCC High"** rather than assumed. See `gcc-high-parity-notes.md` for the cited feature-availability detail.

The Entra application model

When an app integrates with Microsoft Entra, two related objects appear. An `application` (the app registration) is the global definition of the app — its identity, its declared permissions, and its credentials — and it lives in the tenant that authored it. A `servicePrincipal` (the enterprise application) is the local instance of that app in your tenant: it is what you actually assign users to, consent permissions to, and see under Enterprise applications. A multi-tenant SaaS vendor authors one app registration and every customer that signs in gets a service principal. Governance is mostly about service principals — the running instances in your tenant — plus the app registrations you own. This model is identical in GCC High; only the admin URLs and API endpoints change.

Delegated vs. application permissions

Permissions come in two flavors, and the difference is the heart of the risk.

- Delegated permissions act on behalf of a signed-in user and are capped by that user's own rights.

Example: `Mail.Read` delegated lets the app read only the signed-in user's mail.

- Application permissions (app roles) act as the app itself, with no user present, across the whole

tenant. Example: `Mail.Read` as an application permission lets the app read every mailbox in the tenant. These are the dangerous ones — a compromised app with tenant-wide application permissions is a tenant-wide breach.

In the directory, granted delegated permissions appear as `oauth2PermissionGrants`, and granted application permissions appear as `appRoleAssignments` on the service principal.

Consent — who approves the permissions

- User consent: a user clicks Accept on an app's consent prompt and grants delegated permissions

for themselves. Convenient, but it is exactly how consent-phishing works — an attacker sends a link to a malicious app that requests broad delegated scopes (read all your mail, files, send mail as you), and one careless click hands over a durable OAuth token that survives password changes and MFA.

- Admin consent: an administrator grants permissions for the whole organization — required for

application permissions and for high-privilege delegated scopes. The healthy pattern is to restrict user consent to low-risk permissions and route everything else through an admin-consent workflow, so users can request an app and an admin reviews it before it is trusted.

The user-consent settings and the admin-consent workflow are Generally Available in GCC High (see parity notes), so this whole pattern is supported on a sovereign tenant.

The over-permission problem

Apps accumulate more permission than they use. A vendor's app requests broad scopes to cover every possible feature; an internal script is granted `Directory.ReadWrite.All` because it was quicker than scoping it down. Every unused permission is blast radius: if that app or its secret is stolen, the attacker inherits everything it was granted. Governance means finding these over-permissioned apps — especially those with tenant-wide application permissions — and cutting them back to what they need.

Secret and certificate hygiene

An app authenticates with a client secret (`passwordCredentials`) or a certificate (`keyCredentials`). Two failure modes matter: a credential that expires unexpectedly breaks the integration (an outage), and a credential that never expires — or has a very long life, or is one of many piled on the same app — is a standing risk that outlives the people who created it. Governance tracks expiry so nothing breaks by surprise, and prunes stale or excessive credentials so a leaked secret has a short, bounded life. In an ITAR/CUI GCC High tenant this hygiene is doubly important — a leaked long-lived secret is a controlled-data exposure.

What you can control

- Inventory: every enterprise app and service principal, its owners, its app-role assignments,

its OAuth2 delegated grants, and its credentials.

- User-consent policy: block user consent, or allow it only for verified publishers and low-risk

permissions; enable an admin-consent workflow so users can request review.

- Over-permission review: identify apps holding high-privilege application permissions or broad

delegated grants, and remediate the risky ones.

- Credential lifecycle: track secret and certificate expiry, and remove stale or surplus

credentials.

Commercial vs GCC High — what's different

This is the part that trips up teams who try to run the Commercial steps on a sovereign tenant. The governance *actions* are the same; the *addresses* are not. Full cited detail is in `gcc-high-parity-notes.md`; the essentials:

Thing	Commercial	Government (GCC High)
Entra admin center	`https://entra.microsoft.com`	`https://entra.microsoft.us`
Azure portal	`https://portal.azure.com`	`https://portal.azure.us`
Microsoft 365 admin center	`https://admin.microsoft.com`	`https://portal.office365.us` (`admin.microsoft.us`)
Sign-in / authority	`login.microsoftonline.com`	`login.microsoftonline.us`
Microsoft Graph endpoint	`graph.microsoft.com`	`graph.microsoft.us`
Graph PowerShell connect	`Connect-MgGraph` (Global default)	`Connect-MgGraph -Environment USGov`
Graph Explorer (browser API tool)	Supported	**Not supported** — use PowerShell/code

- **You must sign in with a GCC High account on the `.us` portals.** A GCC High credential is not recognized on the Commercial `.com` portals, and vice-versa. Tokens are not interchangeable across clouds. (Cited in parity notes.)
- **The scripts use `-Environment USGov`.** Every script in this package connects to `graph.microsoft.us` via the Microsoft Graph PowerShell SDK's `USGov` environment. Running the Commercial variant of these scripts (Global endpoint) against a GCC High tenant fails to authenticate.
- **Feature parity is close but not total.** Enterprise app governance, consent settings, the admin-consent workflow, PIM, access reviews, and Entra ID Governance are GA in GCC High. A short list of adjacent features lags or is absent (for example Microsoft Entra threat-intelligence risk detection, and Graph Explorer as a browser tool) — see parity notes so you do not build a step on a feature that is not there.

Benefits

- Security — the number-one payoff: risky consents revoked, over-privilege cut, and consent-phishing blocked at the policy level before a user can be tricked — on a sovereign tenant.
- Visibility — a real inventory of what apps exist, who owns them, and what they can do.
- Continuity — no more integrations breaking because a secret expired unnoticed.
- Repeatability — a governance pattern you run on a schedule, not a one-time cleanup.

Limitations (be honest with buyers)

• Revoking a permission or removing a credential can break a legitimate app — every change is staged, validated on a pilot scope, and reversible, and the remediation script defaults to a preview (`-WhatIf`).

• The admin-consent workflow and some risk features need Entra ID P1 or P2 (the Government SKUs are the G-series — see requirements).

• This governs Entra app permissions and consent; it does not replace a full third-party SaaS security review of what each vendor does with the access.

• GCC High feature parity trails Commercial for some adjacent capabilities; never assume a Commercial feature is present without checking the parity notes or the live portal.

The standard deployment model

> *Inventory apps and service principals -> review permissions and consent for over-privilege -> configure the user-consent policy and admin-consent workflow on a pilot scope -> validate -> remediate risky grants and manage credential expiry -> expand tenant-wide. repeatable pattern: Assess -> Configure -> Pilot -> Validate -> Expand.*

2. Requirements Guide — Enterprise App & Service Principal Governance (Government / GCC High)

Verify every requirement before configuring, using the **US sovereign** portals and endpoints. Each entry: Description / Why / Verify / Implement-if-missing. Category-wide prerequisites are in `global-requirements-entra-id-solutions.md`. Cited GCC High feature-availability detail is in `gcc-high-parity-notes.md`.

> *GCC High reminder: sign in at* `https://entra.microsoft.us` *(Entra),* `https://portal.azure.us` *(Azure),* `https://portal.office365.us` *(M365 admin) with a GCC High account. A GCC High credential is not recognized on the Commercial* `.com` *portals. The scripts connect with* `Connect-MgGraph -Environment USGov` *(*`graph.microsoft.us`*).*

2.1 GCC High tenant

Description: An active Microsoft Entra ID tenant in the US Government L4 (GCC High) cloud. Why: All portals, endpoints, and Graph calls target the sovereign `.us` instance; a Commercial tenant uses different URLs and is not interchangeable. Verify: Sign in to `https://entra.microsoft.us` as an administrator; confirm the tenant resolves and that `Connect-MgGraph -Environment USGov` authenticates. Implement: Provision or migrate to a GCC High tenant through your Microsoft/Government partner; this solution does not create the tenant.

2.2 Entra ID licensing (Government SKUs)

Description: Microsoft Entra ID P1 (P2 if you also want risk-based and identity-protection signals), delivered through the US Government plans — typically Microsoft 365 **G3/G5 (GCC High)** or EMS G3/G5, or standalone Entra ID P1/P2 for Government. Why: The admin-consent workflow and some app-governance and risk features need P1+; the core app and service-principal inventory and consent settings work with any tenant, but the review workflow that makes this repeatable needs P1. Verify: `https://portal.office365.us` -> `Billing -> Licenses`; confirm Entra ID P1/P2 (bundled in M365 G3/G5 or EMS G3/G5). Implement: Assign M365 G3/G5, EMS G3/G5, or standalone Entra ID P1/P2 (Government). Note: Entra ID Governance (Access reviews, Entitlement management, PIM, Lifecycle workflows) is GA in GCC High — see parity notes.

2.2a Confirm US Gov feature parity for anything you layer on

Description: A quick check of `gcc-high-parity-notes.md` before you build a step on any adjacent feature. Why: GCC High parity is close but not total. For example, **Microsoft Entra threat intelligence** (an ID Protection risk detection) is not available, and **Graph Explorer** is not supported in GCC High — use the PowerShell scripts instead. Verify: Read the parity notes; for any feature marked "verify for GCC High," confirm it in your `entra.microsoft.us` portal before depending on it. Implement: If a planned adjacent control (for example a specific risk signal) is not in GCC High, substitute the over-permission report and consent policy, which do not depend on it.

2.3 Administrative roles

Description: An operator role that can read and manage apps and consent. `Application Administrator` (or `Cloud Application Administrator`) for day-to-day app governance; `Privileged Role Administrator` or `Global Administrator` only for the settings that change tenant-wide consent policy. RBAC and these roles are available in US Government (see parity notes). Why: Reviewing and remediating app permissions, consent grants, and credentials requires an app-management role; changing the user-consent policy is a tenant-wide, high-privilege action. Verify: `entra.microsoft.us` -> `Roles and administrators`; confirm `Application Administrator` (and a `Global Administrator` available for the consent-policy change). Implement: Assign `Application Administrator` (or `Cloud Application Administrator`); reserve `Global Administrator` for the one-time consent-policy change.

2.4 Consent policy decision

Description: A documented decision on how much users may consent to on their own — block user consent, or allow it only for verified publishers and low-risk delegated permissions — and whether to enable the admin-consent workflow. Why: It is the single most consequential control for stopping consent-phishing. The default in many tenants is far too permissive. Verify: `entra.microsoft.us` -> `Entra ID -> Enterprise apps -> Consent and permissions -> User consent settings`; confirm the current setting is a deliberate choice, not the default. Implement: Default to "Allow user consent for apps from verified publishers, for selected permissions" (low-risk delegated scopes only); enable the admin-consent workflow so users can request apps for review. Decide in Implementation Step 4.

2.5 App inventory approach

Description: A repeatable way to enumerate every enterprise app and service principal with its app-role assignments, delegated grants, owners, and credentials — via US Gov Graph. Why: You cannot govern what you have not inventoried; the tenant almost always holds more apps than anyone expects. Verify: Run `APPGOVG-Discovery.ps1`; confirm it connects with `-Environment USGov` and returns service principals with their app-role assignments, OAuth2 grants, and credential expiry. Implement: Install the Microsoft Graph PowerShell SDK (see 2.7) and run `APPGOVG-Discovery.ps1`.

2.6 Governance scope group (pilot)

Description: A pilot scope for the consent-policy and workflow rollout — a security group of representative users named `App Governance Consent Scope - Pilot - Pilot`, plus a reviewer group `App Governance Consent Reviewers - Tenant - Steady` for the admin-consent workflow. Why: Every change is validated on a pilot before the tenant. The user-consent policy is tenant-wide, so the pilot gate is the admin-consent workflow rollout and the remediation of a first batch of apps — not a per-user policy split. Verify: The groups exist with representative members; run `APPGOVG-Set-ScopeTargeting.ps1` to preview. Implement: Create the groups with `APPGOVG-Set-ScopeTargeting.ps1` (see Implementation Step 1).

2.7 Microsoft Graph PowerShell SDK (US Gov)

Description: The Microsoft Graph PowerShell SDK on the operator workstation, with consent to the read scopes the scripts use, connecting to the **USGov** environment. Why: Discovery, validation, and reporting read service principals, applications, app-role assignments, OAuth2 grants, credentials, and the authorization policy through Microsoft Graph — and in GCC High that must be the `graph.microsoft.us` endpoint, because Graph Explorer is not supported. Verify: `Get-Module -ListAvailable Microsoft.Graph.Authentication` returns a version; a test `Connect-MgGraph -Environment USGov` succeeds and `(Get-MgContext).Environment` reads `USGov`. Implement: `Install-Module Microsoft.Graph -Scope CurrentUser`; the scripts request the least privilege they need (`Application.Read.All`, `Directory.Read.All`, `Policy.Read.All`, and — only for remediation — `Application.ReadWrite.All` / `DelegatedPermissionGrant.ReadWrite.All`) against USGov.

2.8 Break-glass accounts

Description: Two cloud-only emergency access accounts excluded from broad policies. Why: A consent-policy or app-permission change could interfere with admin tooling; break-glass accounts restore access. Verify: `entra.microsoft.us` -> `Users`; two cloud-only `@<tenant>.onmicrosoft.us` admins exist (see global requirement G4; note the `.us` onmicrosoft suffix in GCC High). Implement: Create them, exclude from broad policies, store credentials securely, and monitor sign-ins.

2.9 Network egress to US Gov endpoints

Description: Outbound HTTPS (443) from the operator workstation to the GCC High service endpoints. Why: The scripts and portals reach `login.microsoftonline.us`, `graph.microsoft.us`, `entra.microsoft.us`, and `portal.azure.us`; a proxy/firewall that only allows the Commercial `.com` endpoints will block them. Verify: From the workstation, reach `https://login.microsoftonline.us` and `https://graph.microsoft.us`. Implement: Allow the Microsoft 365 U.S. Government GCC High URL/IP ranges through the proxy/firewall (see M365 US Gov GCC High endpoints, cited in parity notes).

Checklist

- GCC High (US Gov L4) tenant confirmed — signs in at `entra.microsoft.us`
- Entra ID P1/P2 licensing assigned via Government SKUs (M365/EMS G3/G5)
- GCC High parity checked for any adjacent feature (`gcc-high-parity-notes.md`)
- `Application Administrator` role assigned; `Global Administrator` available for consent-policy change
- User-consent policy decision documented (verified-publisher, low-risk scopes) + admin-consent workflow decision
- App inventory produced — `APPGOVG-Discovery.ps1` (connects `-Environment USGov`)
- Governance scope + reviewer groups created — `APPGOVG-Set-ScopeTargeting.ps1`
- Microsoft Graph PowerShell SDK installed and consented; `Connect-MgGraph -Environment USGov` works
- Two break-glass accounts created and excluded from broad policies
- Network egress to `login.microsoftonline.us` / `graph.microsoft.us` allowed

3. Implementation Guide — Enterprise App & Service Principal Governance (Government / GCC High)

Follow these steps in order. Most steps are in the **Microsoft Entra admin center for US Government** (`https://entra.microsoft.us`); the inventory and remediation steps use the included scripts, which connect to US Gov Microsoft Graph with `-Environment USGov`.

> *GCC High caveat (read first): sign in to `entra.microsoft.us` with a GCC High account — the Commercial `entra.microsoft.com` will not recognize it. Graph Explorer is **not supported** in GCC High, so every API action here is done via the PowerShell scripts, not the browser API tool. See `gcc-high-parity-notes.md` for citations.*
>
> *Model: inventory first, review permissions and consent for over-privilege, set the user-consent policy and admin-consent workflow, then remediate a first batch of risky apps and manage credential expiry on a pilot scope before expanding tenant-wide. Every change is staged and reversible.*

Step 0 — Confirm you are in the sovereign cloud

1. Browse to `https://entra.microsoft.us` and sign in with your GCC High admin account.
2. On the operator workstation, run `Connect-MgGraph -Environment USGov -NoWelcome` once and confirm

`(Get-MgContext).Environment` returns `USGov`. If it returns `Global`, you are pointed at the Commercial cloud — reconnect with `-Environment USGov`.

1. Confirm licensing and roles per the Requirements Guide (2.2, 2.3).

Step 1 — Create the governance scope and reviewer groups

1. `entra.microsoft.us` -> `Entra ID -> Groups -> All groups -> New group` -> Security ->

`App Governance Consent Scope - Pilot - Pilot` (a few representative users). Create.

1. Create `App Governance Consent Reviewers - Tenant - Steady` — the admins who will review

admin-consent requests. Add the reviewers.

1. Or run `APPGOVG-Set-ScopeTargeting.ps1` (connects `-Environment USGov`) to create and populate the

groups and preview membership.

Step 2 — Inventory every enterprise app and service principal

1. `entra.microsoft.us` -> `Entra ID -> Enterprise apps -> All applications` to see the

service principals in the tenant.

1. Run `APPGOVG-Discovery.ps1`. It exports, per service principal: the app-role assignments

(application permissions), the OAuth2 delegated grants, the owners, and the client-secret and certificate expiry — all read from `graph.microsoft.us`. Save the output — it is your baseline.

Step 3 — Review API permissions and consents for over-privilege

1. For a suspect app: `entra.microsoft.us` -> `Enterprise apps ->` open the app `-> Permissions`.

Review the admin-consented permissions; note any tenant-wide application permissions (for example `Mail.Read`, `Directory.ReadWrite.All`, `Application.ReadWrite.All`).

1. Run `APPGOVG-Report.ps1` to rank apps by risk — those holding high-privilege application

permissions, broad delegated grants, many credentials, or an unknown/absent owner rise to the top.

1. Build a shortlist of over-permissioned and unowned apps to remediate in Step 6.

GCC High caveat: do not rely on the **Microsoft Entra threat intelligence** risk signal to flag apps — it is not available in GCC High (parity notes). The over-permission report is your primary signal here.

Step 4 — Configure the user-consent policy

1. `entra.microsoft.us` -> `Entra ID -> Enterprise apps -> Consent and permissions ->

User consent settings`.

1. Set `User consent for applications` to `Allow user consent for apps from verified publishers, for

selected permissions (`low-risk delegated scopes only). Do not leave it on` Allow user consent for all apps`.

1. Under `Consent and permissions -> Permission classifications`, confirm only low-risk delegated

permissions (for example `openid`, `profile`, `User.Read`, `offline_access`) are classified as low.

1. Save. This is the tenant-wide control that stops consent-phishing.

GCC High caveat: the *capability* is GA in GCC High, but if a sub-option looks different from a Commercial screenshot, trust the live `entra.microsoft.us` portal — do not assume a Commercial-only option exists here (parity notes, "verify for GCC High").

Step 5 — Enable the admin-consent workflow

1. `entra.microsoft.us` -> `Entra ID -> Enterprise apps -> Consent and permissions ->

Admin consent settings`.

1. Set `Users can request admin consent to apps they are unable to consent to` to `Yes`.

2. Add `App Governance Consent Reviewers - Tenant - Steady` as the reviewers who receive and act on

requests; set the notification and request-expiry options. Save. Note: only reviewers holding a role able to grant tenant-wide consent (for example Global Administrator) can approve requests for Microsoft Graph application permissions.

Step 6 — Remediate risky grants (staged)

1. Preview first: run `APPGOVG-Remediation.ps1 -AppId your-app-id` with its default `-WhatIf`. It

reports the risky OAuth2 delegated grants and app-role assignments it would revoke — nothing is changed. (Connects `-Environment USGov`.)

1. Review the preview against the Step 3 shortlist with the app owners; confirm each app can lose the

grant without breaking a needed function.

1. Re-run with `-WhatIf:$false` scoped to the confirmed apps to revoke the risky grants. To revoke a

single app in the portal instead: `Enterprise apps ->` open the app `-> Permissions ->` remove the permission, or `Properties -> Delete` to remove an unused app entirely.

Step 7 — Manage secret and certificate expiry

1. Run `APPGOVG-Report.ps1` (expiring-secret section) to list credentials expiring soon and

credentials with no expiry or an excessive lifetime.

1. For an app you own: `entra.microsoft.us` -> `App registrations ->` open the app

`-> Certificates & secrets`; rotate the secret or certificate before expiry and remove stale or surplus credentials.

1. Prefer a certificate over a long-lived client secret for app-only authentication — especially on a CUI/ITAR tenant, where a leaked long-lived secret is a controlled-data exposure.

Step 8 — Validate on the pilot

1. Run `APPGOVG-Validation.ps1` -> confirms the user-consent policy is set to verified-publisher / low-risk, the admin-consent workflow is enabled with reviewers, and (with `-CheckAppId`) that a pilot-scope app no longer holds a revoked risky grant. It reads back from `graph.microsoft.us`.

1. Have a pilot user attempt to consent to a high-risk app -> the attempt should be blocked and route to an admin-consent request. Complete the Testing Guide.

Step 9 — Expand tenant-wide

1. With the policy validated, work the full Step 3 shortlist in batches — remediate the remaining over-permissioned and unowned apps with `APPGOVG-Remediation.ps1` (preview, confirm, apply).

1. Put credential expiry on a schedule: re-run `APPGOVG-Report.ps1` monthly and rotate ahead of expiry.

1. Re-run `APPGOVG-Discovery.ps1` and `APPGOVG-Report.ps1` on a recurring cadence so new apps and

grants are caught as they appear.

Order of operations

```
Confirm USGov -> scope + reviewer groups -> inventory -> permission/consent review -> user-consent policy -> admin-consent workflow -> remediate risky grants -> credential expiry -> validate -> expand
```

Rollback

• User-consent policy: revert `User consent settings` to the previous option in `Consent and permissions` (change is immediate and reversible).

• Admin-consent workflow: set `Users can request admin consent...` back to `No`.

• Revoked grant: re-add the permission on the app's `Permissions` blade (or re-grant admin consent); because remediation is confirmed per app from a `-WhatIf` preview, only reviewed apps are ever changed.

5. Testing Guide — Enterprise App & Service Principal Governance (Government / GCC High)

Run after the pilot configuration applies (Implementation Step 8) and on the recurring governance cadence. All scripts connect to US Gov Microsoft Graph (`-Environment USGov`); all portal checks are at `https://entra.microsoft.us`.

Test 0 — You are in the sovereign cloud

Do: Run `Connect-MgGraph -Environment USGov -NoWelcome`, then `(Get-MgContext).Environment`. Expected: Returns `USGov` (not `Global`). If `Global`, you are on the Commercial cloud — reconnect before trusting any other test.

Test 1 — Inventory is complete

Do: Run `APPGOVG-Discovery.ps1`. Expected: Every enterprise app and service principal is listed with its app-role assignments, OAuth2 delegated grants, owners, and credential expiry — a usable baseline.

Test 2 — Over-permission report is accurate

Do: Run `APPGOVG-Report.ps1` and spot-check one flagged app in the portal (`entra.microsoft.us -> Enterprise apps ->` app `-> Permissions`). Expected: The high-privilege application permissions and broad delegated grants the report flags match what the portal shows.

Test 3 — User-consent policy is set

Do: Run `APPGOVG-Validation.ps1`, or check `Consent and permissions -> User consent settings`. Expected: User consent is `Allow user consent for apps from verified publishers, for selected permissions` — not `Allow user consent for all apps`.

Test 4 — Consent-phishing is blocked

Do: As a pilot user, attempt to consent to a high-risk / unverified test app requesting broad delegated scopes. Expected: The self-consent is blocked and the user is offered an admin-consent request instead.

Test 5 — Admin-consent workflow works

Do: The pilot user submits the request from Test 4. Expected: A reviewer in `App Governance Consent Reviewers - Tenant - Steady` receives the request and can approve or deny it (approval of Microsoft Graph application permissions requires a role that can grant tenant-wide consent).

Test 6 — Risky grant remediation (preview then apply)

Do: Run `APPGOVG-Remediation.ps1 -AppId your-app-id` (default `-WhatIf`) against a confirmed over-permissioned app, then re-run with `-WhatIf:$false`. Expected: The preview lists exactly the grant it will revoke; after applying, `APPGOVG-Validation.ps1 -CheckAppId your-app-id` confirms the grant is gone and the app no longer holds it.

Test 7 — Credential expiry surfaced

Do: Run `APPGOVG-Report.ps1` (expiring-secret section). Expected: Secrets and certificates expiring soon, and any with no expiry or an excessive lifetime, are listed with their app and days-to-expiry.

Pass/fail gate

Gate	Requirement
Pilot pass	Test 0 confirms USGov; Tests 1-5 pass; a first over-permissioned app is remediated (Test 6) and validated
Expand pass	Tests 1-3, 7 pass tenant-wide; the risky-app shortlist is worked and credential expiry is on a schedule

6. Troubleshooting Guide — Enterprise App & Service Principal Governance (Government / GCC High)

Use `APPGOVG-Discovery.ps1` for inventory, `APPGOVG-Report.ps1` for over-permission and expiring-secret status, `APPGOVG-Validation.ps1` for policy state, and `APPGOVG-Remediation.ps1` (preview first) for safe fixes. All connect to US Gov Microsoft Graph (`-Environment USGov`); all portal steps are at `https://entra.microsoft.us`.

A. Wrong cloud / GCC High connection problems (check this first)

Symptom	Cause	Remediation
`Connect-MgGraph` signs in but returns no tenant data, or the wrong tenant	Connected to the Commercial Global endpoint instead of USGov	Reconnect with `Connect-MgGraph -Environment USGov`; confirm `(Get-MgContext).Environment` is `USGov`
Sign-in fails / "account not recognized" on the portal	Used the Commercial `entra.microsoft.com` with a GCC High account	Use `https://entra.microsoft.us`; GCC High credentials are only valid on `.us` endpoints (tokens are not interchangeable across clouds)
Redirect or "wrong authority" error from Graph PowerShell	SDK defaulted to `login.microsoftonline.com`	Ensure `-Environment USGov` (authority `login.microsoftonline.us`, Graph `graph.microsoft.us`); update the Microsoft.Graph module if old
You wanted to poke the API in Graph Explorer	Graph Explorer is **not supported** in GCC High	Use the PowerShell scripts (they target `graph.microsoft.us`) — there is no browser Graph Explorer for the sovereign cloud

B. Discovery or report returns nothing / errors

Symptom	Cause	Remediation
`Connect-MgGraph` fails or scopes are missing	SDK not installed, or consent to read scopes not granted	`Install-Module Microsoft.Graph -Scope CurrentUser`; re-run with `-Environment USGov` and accept the `Application.Read.All`, `Directory.Read.All`, `Policy.Read.All` scopes
Empty results	Signed in to the wrong tenant/cloud, or an app-read role missing	Confirm USGov (section A) and the tenant; ensure the operator holds `Application Administrator` / `Cloud Application Administrator`
Owners show as blank	The app genuinely has no owner	Expected — flag it; assign an owner (see C)

C. Unowned or unknown apps

- An app with no owner and unclear purpose is the highest-priority review item. Check its

`passwordCredentials` / `keyCredentials` and recent sign-ins, identify the vendor from the app name or publisher, assign an owner, and if it is unused remove it (`entra.microsoft.us -> Enterprise apps ->` app `-> Properties -> Delete`).

D. A legitimate app broke after remediation

Symptom	Cause	Remediation
Integration fails after a grant was revoked	The revoked permission was actually in use	Re-add the permission on the app's `Permissions` blade (or re-grant admin consent); revert is immediate. Always run `APPGOVG-Remediation.ps1` in `-WhatIf` first and confirm with the owner
App-only job fails at a certain date	The client secret or certificate expired	Rotate the credential in `App registrations -> Certificates & secrets`; track future expiry with `APPGOVG-Report.ps1`

E. User-consent policy not taking effect

- After changing `User consent settings`, confirm you saved and that you edited the tenant-wide

setting under `Consent and permissions` on `entra.microsoft.us`, not a single app. Run `APPGOVG-Validation.ps1` to read back the effective `authorizationPolicy` consent setting; the change is tenant-wide and near-immediate.

F. Admin-consent requests not reaching reviewers

- Confirm `Admin consent settings -> Users can request admin consent...` is `Yes`, the reviewer group

`App Governance Consent Reviewers - Tenant - Steady` is set as reviewers, and notifications are enabled. Reviewers must hold a role able to grant tenant-wide consent (for example Global Administrator) to approve Microsoft Graph application permissions.

G. Consent-phishing still succeeding

- If a user still consented to a broad app, a high-risk delegated permission is likely classified as

low-risk. Review `Consent and permissions -> Permission classifications` and remove anything but genuinely low-risk scopes (for example keep only `openid`, `profile`, `User.Read`, `offline_access`). Revoke the bad grant with `APPGOVG-Remediation.ps1`.

H. A feature you saw in Commercial is missing in GCC High

- GCC High parity is close but not total. Confirmed gaps that affect this area: **Microsoft Entra

threat intelligence **(an ID Protection risk detection) is not available,** Graph Explorer** is not supported, and Defender for Cloud Apps app-governance features can trail Commercial by up to ~3 months. Check `gcc-high-parity-notes.md`; for anything marked "verify for GCC High," trust the live `entra.microsoft.us` portal over a Commercial screenshot. Do not build a runbook step on a feature that is not present.

I. Diagnostics quick reference (US Gov)

```
Connect-MgGraph -Environment USGov -Scopes 'Application.Read.All','Directory.Read.All','Policy.Read.All' -
(Get-MgContext).Environment   # must be USGov
# effective user-consent policy
(Invoke-MgGraphRequest -Method GET -Uri '/v1.0/policies/authorizationPolicy').defaultUserRolePermissions
# an app's granted application permissions and delegated grants (use your service principal object id)
Get-MgServicePrincipalAppRoleAssignment -ServicePrincipalId your-sp-object-id
Get-MgOauth2PermissionGrant -Filter "clientId eq 'your-sp-object-id'"
# an app's credentials
(Get-MgServicePrincipal -ServicePrincipalId your-sp-object-id).PasswordCredentials
(Get-MgServicePrincipal -ServicePrincipalId your-sp-object-id).KeyCredentials
```

J. When to escalate

If an app cannot be attributed to any owner or vendor, holds tenant-wide application permissions, and shows unexpected sign-in activity, treat it as a potential compromise: revoke its grants and credentials, disable the service principal (`AccountEnabled` false), and engage your incident-response process. On a CUI/ITAR tenant, follow your spill/incident procedure — an over-permissioned compromised app may constitute a controlled-data exposure.

7. Best Practices — Enterprise App & Service Principal Governance (Government / GCC High)

• Always confirm the sovereign cloud first. Sign in at `entra.microsoft.us` and connect Graph with `-Environment USGov`. A single Commercial-endpoint call against a GCC High tenant fails silently or authenticates the wrong tenant — verify `(Get-MgContext).Environment` is `USGov` before you act.

• Restrict user consent — the single most important control. Allow user consent only for verified publishers and low-risk delegated permissions; never leave it on `Allow user consent for all apps`. This is what stops consent-phishing before a user can be tricked.

• Turn on the admin-consent workflow. Blocking user consent without a request path just drives shadow IT; let users request an app and have a named reviewer group approve or deny it.

• Treat application permissions as tenant-wide breach potential. Scrutinize every app holding application permissions (especially `*.ReadWrite.All`, `Directory.*`, `Mail.*`, full-mailbox or full-directory scopes); grant them only to apps that truly need them, and only by admin consent. On a CUI/ITAR tenant this blast radius is controlled-data exposure — hold the line harder.

• Enforce least privilege. Cut every app back to the permissions it actually uses; unused permission is pure blast radius if the app or its secret is stolen.

• Every app needs an owner. Unowned apps are how sprawl becomes risk — assign an owner to each app so someone can answer for it and rotate its credentials.

• Prefer certificates over long-lived secrets. Use a certificate for app-only authentication, keep secret lifetimes short, and never let a credential sit with no expiry — a leaked long-lived secret on a sovereign tenant is a data-spill risk.

• Track expiry proactively. Run the report on a schedule and rotate before expiry so integrations never break by surprise; prune stale and surplus credentials so a leaked secret has a short life.

• Stage and preview every removal. Revoking a permission or deleting an app can break a legitimate integration — always run remediation in `-WhatIf` preview first, confirm with the owner, then apply to the reviewed apps only.

• Do not depend on features that are not in GCC High. Microsoft Entra threat intelligence risk detection is not available, and Graph Explorer is not supported — use the over-permission report and the PowerShell scripts. Check `gcc-high-parity-notes.md` before building any step on an adjacent feature.

• Make it recurring, not one-time. New apps and grants appear constantly; re-run discovery and the report on a cadence so governance is a standing process.

• Document the estate. Record each app's owner, purpose, permissions, and credential-rotation date

for audits and repeatability — GCC High tenants are frequently audited (CMMC/ITAR), and this record is your evidence.

8. Deliverables Summary — Enterprise App & Service Principal Governance (Government / GCC High)

Documentation

Document	Purpose
Executive Summary	Business case and what is in the box — GCC High edition
1. Solution Overview	The Entra app model, delegated vs. application permissions, consent and consent-phishing, over-permission, secret/certificate lifecycle, plus **Commercial vs GCC High — what's different**
GCC High Parity Notes	Cited US Gov endpoints, confirmed GA features, and confirmed parity gaps; "verify for GCC High" items
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing, in US Gov terms (GCC High tenant, G-series licensing, USGov Graph)
Global Requirements (Entra ID Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps using the `entra.microsoft.us` portals with GCC-High caveats inline (inventory, permission/consent review, user-consent policy, admin-consent workflow, remediate, credential expiry, validate, expand)
5. Testing Guide	Validation procedures with pass/fail gates, including a "you are in USGov" gate
6. Troubleshooting Guide	Field issues and remediation, with GCC-High-specific gotchas
7. Best Practices	Field-proven recommendations for a sovereign tenant

PowerShell scripts (all connect `-Environment USGov` → `graph.microsoft.us`)

Script	What it does
`APPGOVG-Discovery.ps1`	Inventories enterprise apps and service principals with app-role assignments, OAuth2 delegated grants, owners, and credential expiry via US Gov Microsoft Graph (read-only)

Script	What it does
`APPGOVG-Validation.ps1`	Confirms the user-consent policy, admin-consent workflow, and that remediated apps no longer hold risky grants (read-only)
`APPGOVG-Report.ps1`	Over-permission and expiring-secret report — ranks apps by permission risk and lists credentials expiring or with no expiry (read-only)
`APPGOVG-Remediation.ps1`	Safely revokes risky OAuth2 grants and app-role assignments; `-WhatIf` by default
`APPGOVG-Set-ScopeTargeting.ps1`	Builds and previews the governance consent-scope and reviewer groups

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP, pre-wired for US Gov Graph (`-Environment USGov`).
- A repeatable process: Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

Every enterprise app and service principal inventoried and governed on GCC High — right portals, right endpoints, parity gaps handled — over-permissioned grants and risky consents remediated, user consent controlled, and app secrets and certs on a managed lifecycle — closing a favorite attacker foothold, proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

APPGOVG-Discovery.ps1

Purpose: Read-only inventory of every service principal (enterprise application) in a US Government (GCC High) tenant. Per app it reports the application permissions granted via appRoleAssignments, the delegated permissions granted via oauth2PermissionGrants, the owners, and the client-secret (passwordCredentials) and certificate (keyCredentials) expiry. This is your governance baseline. Connects to US Gov Microsoft Graph (graph.microsoft.us). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPGOVG-Discovery.ps1 Optional: -CsvPath .\appgovg-inventory.csv to export a flat CSV. Graph scopes: Application.Read.All, Directory.Read.All, Policy.Read.All (read-only); connects with -Environment USGov. EXPECTED RESULTS A per-service-principal listing of application permissions, delegated grants, owners, and credential expiry. No changes are made (pass = inventory returned; fail = empty/auth error). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #> [CmdletBinding()] param([string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\APPGOVG-Discovery.ps1 Optional: -CsvPath .\appgovg-inventory.csv to export a flat CSV. Graph scopes: Application.Read.All, Directory.Read.All, Policy.Read.All (read-only); connects with -Environment USGov. EXPECTED RESULTS A per-service-principal listing of application permissions, delegated grants, owners, and credential expiry. No changes are made (pass = inventory returned; fail = empty/auth error). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #> [CmdletBinding()] param([string]$CsvPath)

APPGOVG-Remediation.ps1

Purpose: Removes risky permissions from a named, confirmed over-permissioned app on a US Government (GCC High) tenant: its delegated oauth2PermissionGrants and/or its application-permission appRoleAssignments. Intended for apps you have already reviewed from APPGOVG-Report.ps1 and confirmed with the owner. Defaults to -WhatIf, so a plain run only PREVIEWS what it would revoke and changes nothing. Connects to US Gov Microsoft Graph (graph.microsoft.us). HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\APPGOVG-Remediation.ps1 -AppId your-app-id Apply after review: powershell -ExecutionPolicy Bypass -File .\APPGOVG-Remediation.ps1 -AppId your-app-id -WhatIf:$false Scope: -RevokeDelegated / -RevokeAppRoles (default: both). Graph scopes: Application.ReadWrite.All, DelegatedPermissionGrant.ReadWrite.All, Directory.Read.All; connects with -Environment USGov. EXPECTED RESULTS In preview: the exact delegated grants and app-role assignments that WOULD be revoked. When applied: those grants/assignments are removed; re-run APPGOVG-Validation.ps1 -CheckAppId to confirm. Reversible by re-granting the permission in the portal. .NOTES White-label — the repeatable way. Write action — preview (-WhatIf) by default. US Gov Graph endpoints. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [string]$AppId, [switch]$RevokeDelegated, [switch]$RevokeAppRoles)

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\APPGOVG-Remediation.ps1 -AppId your-app-id Apply after review: powershell -ExecutionPolicy Bypass -File .\APPGOVG-Remediation.ps1 -AppId your-app-id -WhatIf:$false Scope: -RevokeDelegated / -RevokeAppRoles (default: both). Graph scopes: Application.ReadWrite.All, DelegatedPermissionGrant.ReadWrite.All, Directory.Read.All; connects with -Environment USGov.

EXPECTED RESULTS In preview: the exact delegated grants and app-role assignments that WOULD be revoked. When applied: those grants/assignments are removed; re-run APPGOVG-Validation.ps1 -CheckAppId to confirm. Reversible by re-granting the permission in the portal. .NOTES White-label — the repeatable way. Write action — preview (-WhatIf) by default. US Gov Graph endpoints. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [string]$AppId, [switch]$RevokeDelegated, [switch]$RevokeAppRoles)

APPGOVG-Report.ps1

Purpose: Read-only risk report over every service principal in a US Government (GCC High) tenant. It (1) ranks apps by permission risk — those holding high-privilege application permissions (appRoleAssignments to *.ReadWrite.All, Directory.*, Mail.*, full-mailbox/full-directory scopes), broad delegated grants (oauth2PermissionGrants), no owner, or many credentials — and (2) lists client secrets (passwordCredentials) and certificates (keyCredentials) that are expired, expiring within a window, or have no expiry / an excessive lifetime. Connects to US Gov Microsoft Graph (graph.microsoft.us). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPGOVG-Report.ps1 Optional: -ExpiryWindowDays 30 (default) to set the "expiring soon" window. Graph scopes: Application.Read.All, Directory.Read.All (read-only); connects with -Environment USGov. EXPECTED RESULTS A ranked over-permission table and an expiring-secret list with app, credential type, and days-to-expiry. No changes are made. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #> [CmdletBinding()] param([int]$ExpiryWindowDays = 30)

How to run: powershell -ExecutionPolicy Bypass -File .\APPGOVG-Report.ps1 Optional: -ExpiryWindowDays 30 (default) to set the "expiring soon" window. Graph scopes: Application.Read.All, Directory.Read.All (read-only); connects with -Environment USGov. EXPECTED RESULTS A ranked over-permission table and an expiring-secret list with app, credential type, and days-to-expiry. No changes are made. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #> [CmdletBinding()] param([int]$ExpiryWindowDays = 30)

APPGOVG-Set-ScopeTargeting.ps1

Purpose: Creates (or previews) the two security groups this solution targets on a US Government (GCC High) tenant: the pilot consent scope "App Governance Consent Scope - Pilot - Pilot" and the admin-consent reviewer group "App Governance Consent Reviewers - Tenant - Steady". Use the reviewer group in the admin-consent workflow (Implementation Step 5) and the pilot scope to stage the rollout. By default it PREVIEWS (no changes); pass -Create to create groups that do not yet exist. Connects to US Gov Microsoft Graph (graph.microsoft.us). HOW TO RUN Preview which groups exist / would be created (read-only): powershell -ExecutionPolicy Bypass -File .\APPGOVG-Set-ScopeTargeting.ps1 Create the groups if missing: powershell -ExecutionPolicy Bypass -File .\APPGOVG-Set-ScopeTargeting.ps1 -Create Graph scopes: Group.ReadWrite.All, Directory.Read.All (Group.Read.All in preview); connects with -Environment USGov. EXPECTED RESULTS In preview: whether each named group already exists and its member count. With -Create: the missing groups are created (empty) and their object IDs returned so you can add members. .NOTES White-label — the repeatable way. Read-only unless -Create is supplied. US Gov Graph endpoints. #> [CmdletBinding()] param([switch]$Create)

How to run: Preview which groups exist / would be created (read-only): powershell -ExecutionPolicy Bypass -File .\APPGOVG-Set-ScopeTargeting.ps1 Create the groups if missing: powershell -ExecutionPolicy Bypass -File .\APPGOVG-Set-ScopeTargeting.ps1 -Create Graph scopes: Group.ReadWrite.All, Directory.Read.All (Group.Read.All in preview); connects with -Environment USGov.

EXPECTED RESULTS In preview: whether each named group already exists and its member count. With -Create: the missing groups are created (empty) and their object IDs returned so you can add members. .NOTES White-label — the repeatable way. Read-only unless -Create is supplied. US Gov Graph endpoints. #> [CmdletBinding()] param([switch]$Create)

APPGOVG-Validation.ps1

Purpose: Read-only check that the governance controls are in place on a US Government (GCC High) tenant: (1) the tenant user-consent policy restricts user consent (verified publishers / low-risk permissions, not "all apps"), read from the authorizationPolicy; (2) the admin-consent workflow is enabled with reviewers; and (3) an optional named app no longer holds a revoked delegated grant or app-role assignment. Connects to US Gov Microsoft Graph (graph.microsoft.us). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPGOVG-Validation.ps1 Optional: -CheckAppId your-app-id to verify a remediated app no longer holds risky grants. Graph scopes: Policy.Read.All, Application.Read.All, Directory.Read.All (read-only); connects with -Environment USGov. EXPECTED RESULTS PASS/WARN lines for the user-consent policy and admin-consent workflow, and — if -CheckAppId is given — the current app-role assignments and delegated grants for that app. No changes are made. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #> [CmdletBinding()] param([string]$CheckAppId)

How to run: powershell -ExecutionPolicy Bypass -File .\APPGOVG-Validation.ps1 Optional: -CheckAppId your-app-id to verify a remediated app no longer holds risky grants. Graph scopes: Policy.Read.All, Application.Read.All, Directory.Read.All (read-only); connects with -Environment USGov. EXPECTED RESULTS PASS/WARN lines for the user-consent policy and admin-consent workflow, and — if -CheckAppId is given — the current app-role assignments and delegated grants for that app. No changes are made. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #> [CmdletBinding()] param([string]$CheckAppId)